

Chapter 5

ASSEMBLY PROGRAMING OF ARM7

INTRODUCTION

- ❑ Assembly Language Programming (ALP) is an efficient way of using the instruction set of a processor.
- ❑ Here coding is done in a symbolic language which is generally referred to as "mnemonics".
- ❑ These mnemonic are directly converted into machine language and are executed by the processor.
- ❑ Assembler: Tool that does the translation from assembly language to machine language.
- ❑ Means its process is one-to-one process such that, each mnemonic gets translated to only a specific machine code.

5.1 EMBEDDED PROGRAM DEVELOPMENT

- ❑ ARM is an embedded processor and the code which is developed has finally to be 'burned' into its ROM.
- ❑ Code has to be developed, and thoroughly tested and verified to be correct and working.
- ❑ We need a host computer which is a PC, in which the development and testing of the code is done
- ❑ PC runs on an x86 processor, but the program that we are working on is for another processor's Instruction Set Architecture (ISA), that is, ARM.
- ❑ when the assembly process is complete, the binary code that is obtained is of a processor which is not the host computer (x86). So the conversion is termed as 'cross assembly'. If the source program is written in high level language, the translation is called 'cross compiling'.

5.1 EMBEDDED PROGRAM DEVELOPMENT (CONTD..)

- ❑ The host PC should have an IDE (Integrated Development Environment) which is a programming environment that has a code editor, cross compiler and assembler, debugger, simulator and graphical user interface (GUI).
- ❑ The IDE has knowledge of the registers, instruction set, memory and peripheral mapping of the embedded processor for which it is used.
- ❑ IDE of 8051 cannot be used for ARM.
- ❑ Our approach is to use assembly programming for the ARM core, and C programming with peripherals

POPULAR IDES IN USE FOR VARIOUS PROCESSORS.

Table 5.1 A short list of popular IDEs

IDE	MCUs for which they are used	Supplier
Keil RVDK	ARM, 8051	Keil Inc,
Eclipse	ARM	OPEN SOURCE
PSoC Creator	PSoC 3 and 5	CYPRESS SEMICONDUCTORS
PSoc Designer	PSoC 1	CYPRESS SEMICONDUCTORS
IAR	MSP 430, ARM, 8051	IAR SYSTEMS
Code Composer Studio	DSP Processors, MSP 430	TEXAS INSTRUMENTS
MPLAB	PIC	MICROCHIP TECHNOLOGY
AVR STUDIO	AVR MCU	ATMEL CORPORATION
CODEWARRIOR	ARM	FREESCALE SEMICONDUCTORS
VISUAL DSP++	Blackfin DSP processor	Analog Devices

NOTE TO REMEMBER

- ❑ When using the assembler, the results of the program are checked in registers and/or memory.
- ❑ When memory is being used, we may need to choose a specific processor, so that we can define the ROM and RAM spaces.
- ❑ We have chosen NXP's LPC 214x series, which has ROM starting at address 0x00000000 and RAM at 0x40000000.

5.1.1 FEATURES OF ARMV4T ISA

- ❑ The version of ISA used for ARM7 is v4T (the character 'T' means that Thumb instructions are also included).
- ❑ Feature are:
 1. In general, all Instructions are of size 32 bits for ARM and 16 bits for THUMB, though there are a few exceptions.
 2. There is a barrel shifter in the ALU, as shown in Fig. 5.1. This means that one of the operands may be shifted/rotated. This is useful for simplifying some types of operations.
 3. The use of the barrel shifter makes possible the combination of shift and ALU operations in a single instruction.
 4. All operands of data processing instructions are in registers.

5. Only the load and store instructions access memory.
6. Most other processors use condition checking only with branch instructions. In contrast, here we find that many other instructions have conditions appended to it
7. There is a special technique for handling immediate instructions.
8. There are instructions to load/store data in multiple registers.
9. Arithmetic and logical instructions have a three operand format.

SOME POINTS TO NOTE BEFORE WE START ASSEMBLY PROGRAMMING

- The processor can opt to use the big endian or little endian format
- In our discussion here, we choose the little endian format.
- 8-bit (byte), 16-bit(half word) and 32-bit(word) data sizes are possible.
- Data alignment is necessary for efficiency.

5.1.2 ASSEMBLY LANGUAGE FORMAT

❑ A line in assembly language has the following format.

Label	Opcode	Operands	;comments
SAM	Add	R7,R6,R5	;add R6 and R5 & copy sum to R7

- SAM is the label, ADD is the opcode, and R7, R6, R5 are the operands.
- What is written after the semicolon is a comment.

5.2 ARM7 INSTRUCTION SET

The instructions can be classified into the following different types:

- I. Data processing instructions
- II. Load store instructions — single register, multiple register
- III. Branch instructions
- IV. Status register access instructions

□ We will discuss only the first three types.

5.2.1 DATA PROCESSING INSTRUCTIONS

- This set includes move, arithmetic, logical and compare instructions.
- Lets start with the concept of the barrel shifter which is part of the ALU.
- A barrel shifter is a unit which can shift and rotate operands by any amount.
- Figure shows that when there are two source operands in the ALU,
- one of them may be shifted/rotated any number of times
- Note that the maximum size of any register is 32 bits
- so the barrel shifter operations are not needed for shift/rotations beyond 32 times.

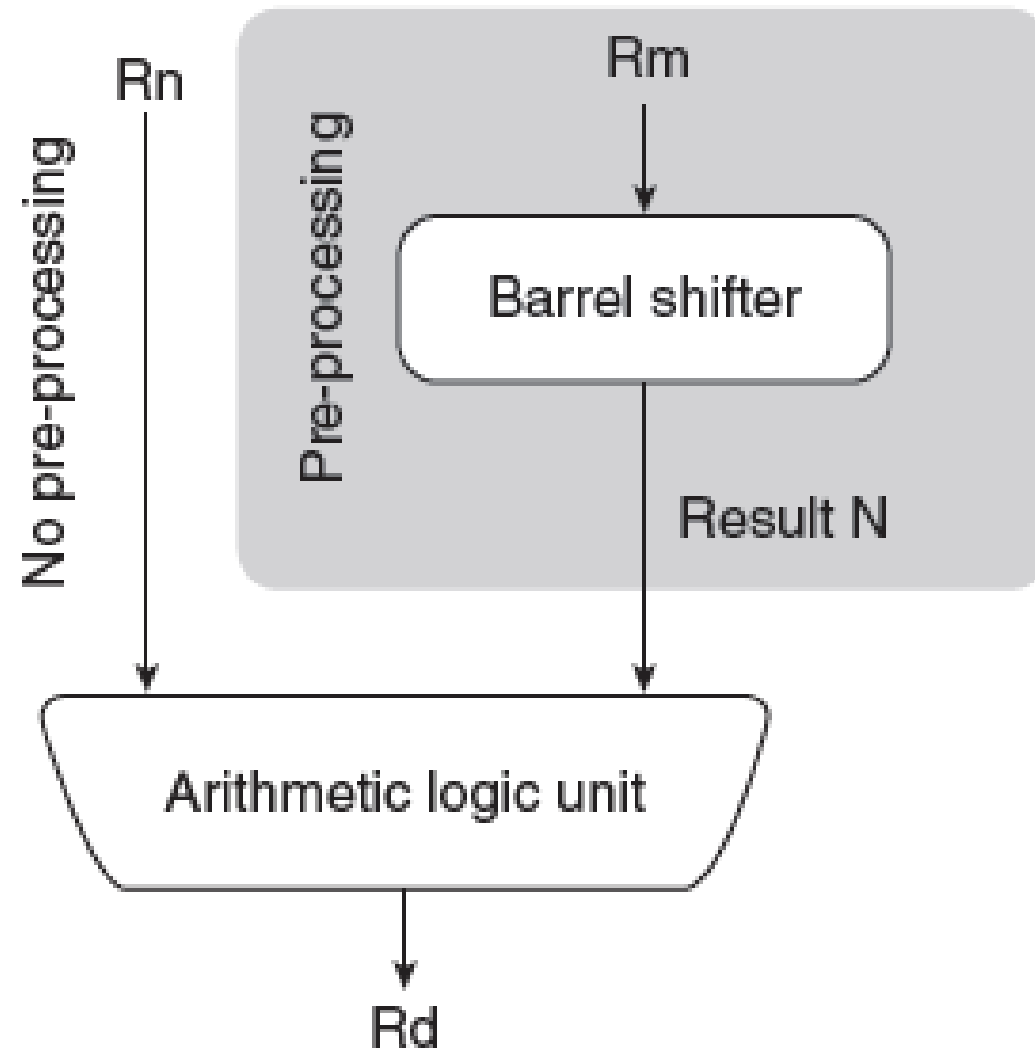


Figure 5.1 Barrel shifter in ALU

Table 5.2 List of shift and rotate operations

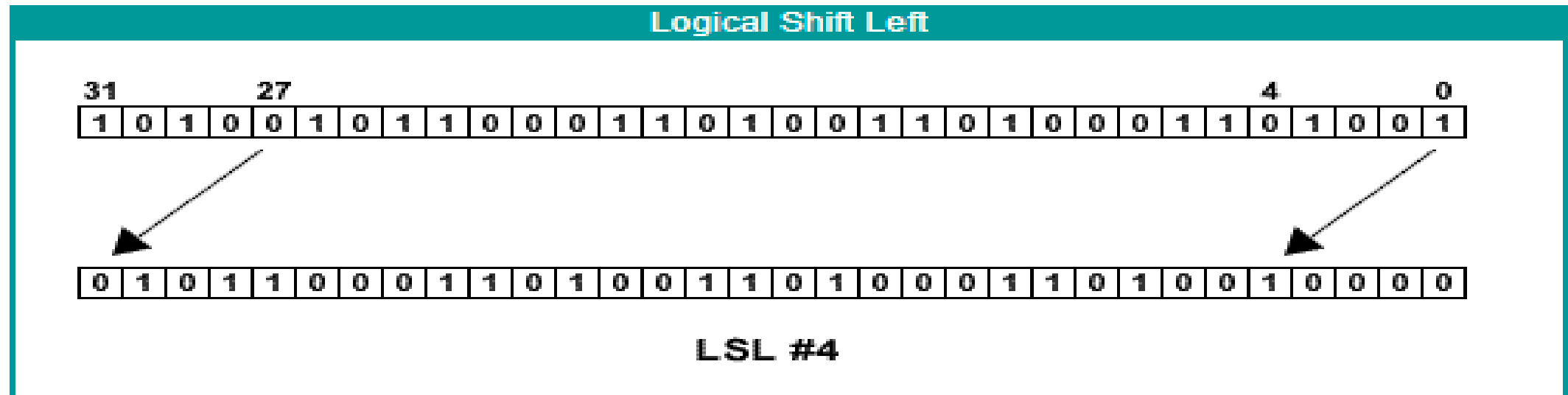
Mnemonic	Description
LSL	logical shift left
LSR	logical shift right
ASR	arithmetic shift right
ROR	rotate right
RRX	rotate right extended

5.2.1.1 LOGICAL SHIFT LEFT (LSL #n)

- When the shift amount is specified in the instruction, it is contained in a 5-bit field which may take any value from 0-31.
- A logical shift left (LSL) takes the contents of the specified register and shifts it left.
- A left shift by one bit position is equivalent to multiplication by 2.
- For example, the effect of LSL #4 causes a multiplication by 2^4 , that is, by 16.
- The least significant bits of the result are filled with zeros, and the high bits of the register are discarded.
- The most significant discarded bit goes into the Carry flag.



LSL is a logical shift /left by 0 to 31 places. The vacated bits at the least significant end of the word are filled with zeros.



EXAMPLE

Consider the following ARM instruction with $r1 = 3$ and $r2 = 5$

```
ADD r0, r1, r2 ; r0 := r1 + r2 which is r0 := 3 + 5 = 8
```

Now try the same instruction with a LSL operand, say LSL #3 (logical shift left 3 places which is equivalent to multiplying by 8 (2^3)):

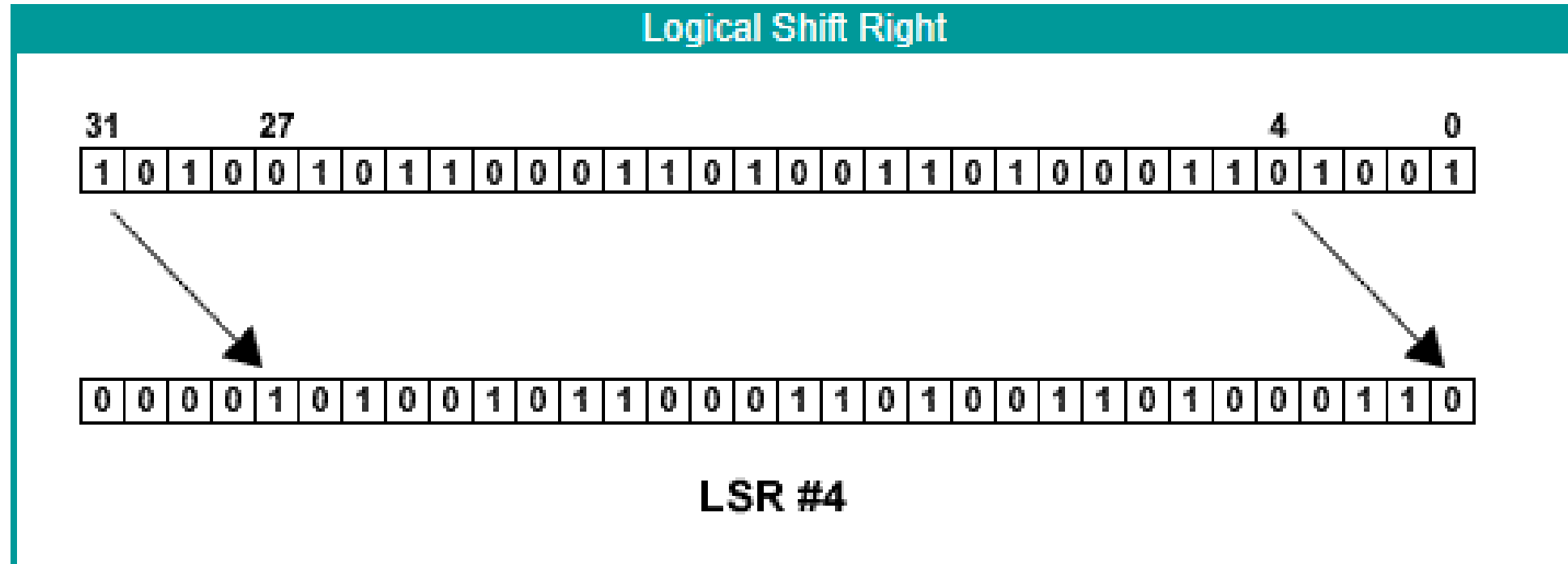
```
ADD r0, r1, r2, LSL #3 ; r0 := r1 + (8 x r2) which is r0 := 3 + (8x5) = 43
```

5.2.1.2 LOGICAL SHIFT RIGHT (LSR #n)

- This is similar to LSL but the shifting is towards right.
- Shifting right by each bit position is equivalent to a division by 2.
- Thus, LSR #5 causes the data in the specified register to be divided by 32.



LSR is a logical shift *right* by 0 to 32 places. The vacated bits at the most significant end of the word are filled with zeros.



Consider a MOV instruction with r1 = 168 and r2 = 3:

MOV	r0, r1, LSR r2	; shift the binary value of 168 3 places to the right
		; 168 = 0000 0000 0000 0000 0000 0000 1010 1000
		; shifted 3 places,
		; becomes 0000 0000 0000 0000 0000 0000 0001 0101
		; r0 := 21

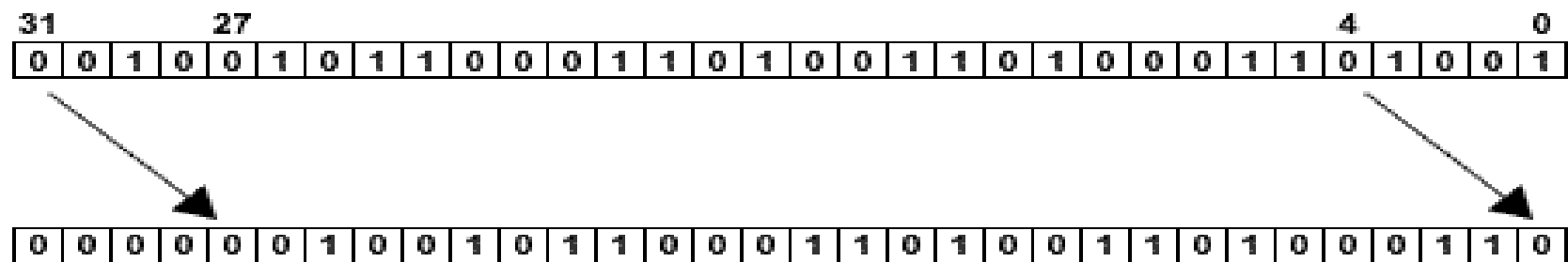
For positive numbers, LSR 3 is the same as dividing by 2^3 (8).

5.2.1.3 ARITHMETIC SHIFT RIGHT (ASR #n)

- An arithmetic shift right (ASR) is similar to logical shift right, except that the high bits are filled with the MSB of the register instead of zeros.
- This preserves the sign in 2's complement notation. Thus, sign extension is done on the data, along with it being divided by 2 for each shift.
- Thus, ASR #4 causes a division by 16

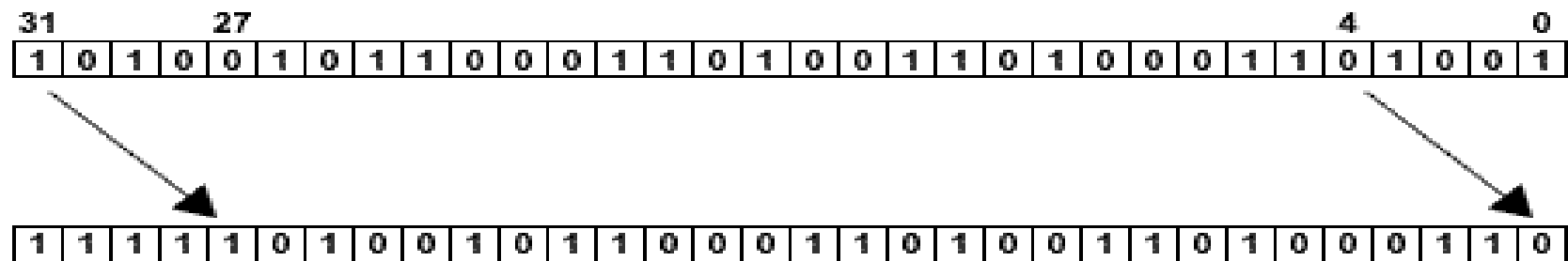


Arithmetic Shift Right Positive Value



Arithmetic Shift Right Positive Value

Arithmetic Shift Right Negative Value



ASR #4 negative value

To show the difference between logical and arithmetic right shifts, consider A LSR and ASR where the value to be shifted is 112 and the shift is 3 places. To keep the mathematics simple, we will use 8-bit numbers.

The bit pattern for 112 is 0111 0000.

Performing LSR #3 transforms the bit pattern to be 0000 1110

Result is 14 : 14 to base 10

Performing ASR #3 again transforms the bit pattern to 0000 1110

Now what happens if the number is -112?

First create the 2's complement bit pattern for -112 by flipping the bit pattern of 112 and adding 1.

0111 0000 becomes (1000 1111 + 1) becomes 1001 0000

Now perform a LSR #3 operation on 1001 0000.

This shifts the bits 3 places to the right and fills in the vacated bits with 0s:

1001 0000 becomes 0001 0010 which is +18 to Base 10

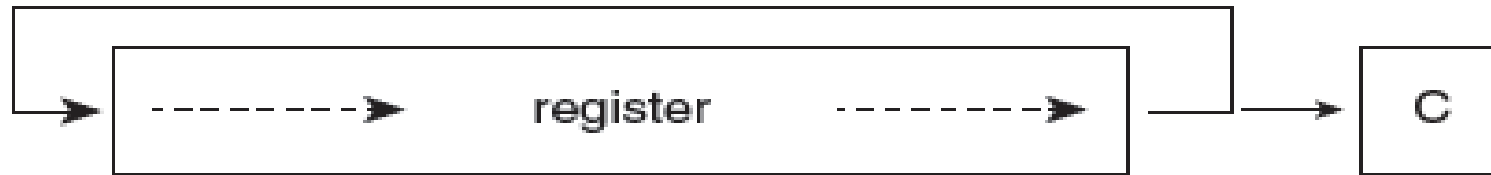
Now instead perform an ASR #3 operation on 1001 0000.

This shifts the bits 3 places to the right and fills the vacated bits with copies of the most significant bit, i.e. 1's.

1001 0000 becomes 1111 0010 which is -14 to base 10.

5.2.1.4 ROTATE RIGHT (ROR #n)

- Right shifting is done and the bits which overflow are reintroduced at the left end of the register.
- The last bit shifted out, is put in the carry flag as well.

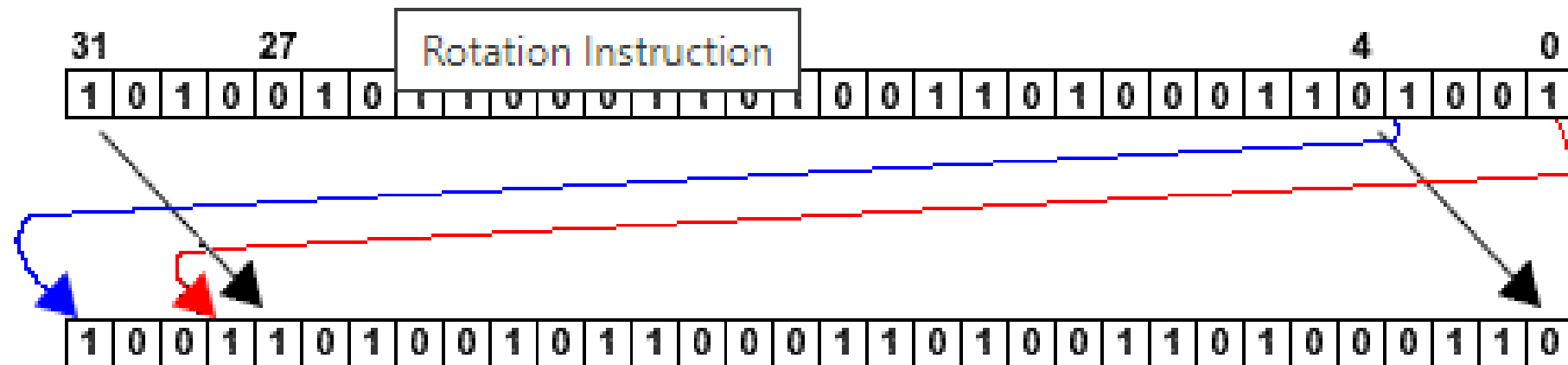


- There is no 'rotate left' instruction, because left rotation by n times can be achieved by rotating to the right $(32 - n)$ times.
- For example, rotating 4 times to the left is achieved by rotating $32 - 4 = 28$ times to the right.

MOV r0, r0, ROR #3 ; rotate value in r0 3 places and put result in r0

MOV r2, r0, ROR r1 ; rotate r0 by value of places in r1 and put result in r2

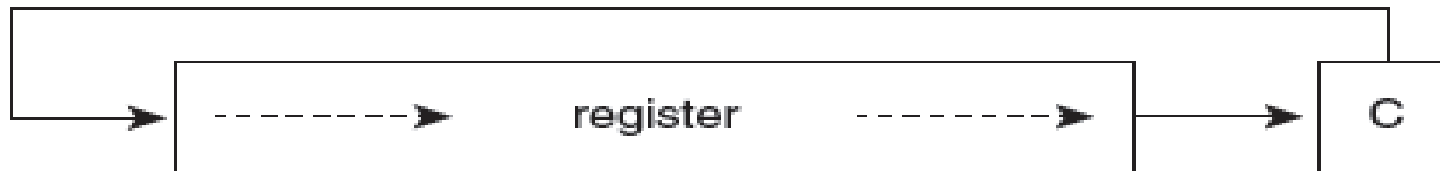
Rotation



ROR #4

5.2.1.5 ROTATE RIGHT EXTENDED (RRX #n)

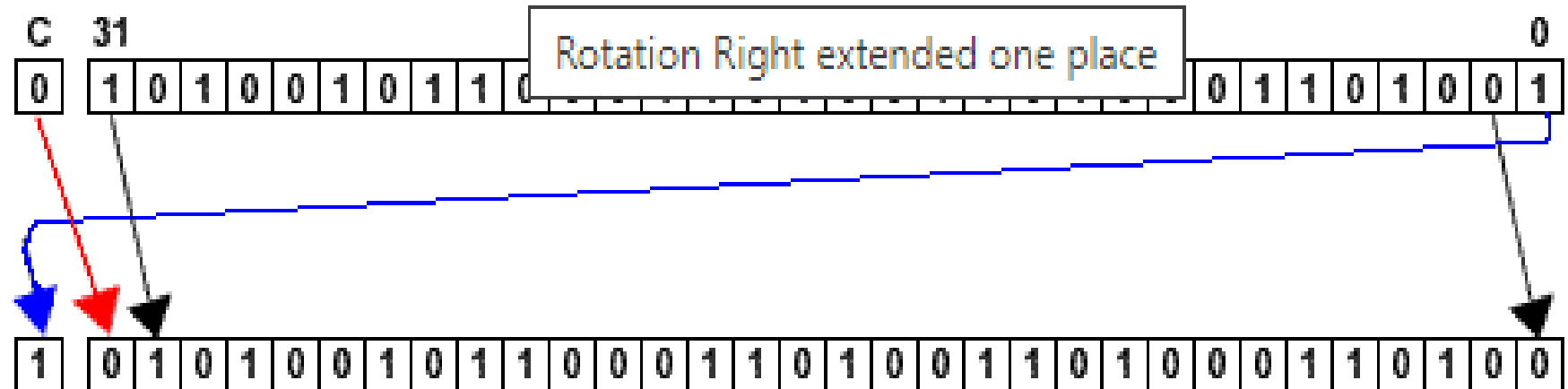
- RRX is a ROR operation with a crucial difference.
- It rotates the number to the right by one place but the original bit 31 is filled by the value of the Carry flag and the original bit 0 is moved into the Carry flag.



- This allows a 33-bit rotation by using both the register and the carry flag.

MOV r0, r0, RRX ; rotate right extended and put result back into r0

Rotation Right extended one place



RRX

PROBLEMS

Find the results of the following operations given the contents of
R1=0x0089EF32, R2=0x80456730, R3=0x8

1. LSL R1,#8
2. LSR R1,#12
3. ASR R2,R3
4. ROR R2,R3

ANSWERS

1. LSL R1,#8

R1=0000 0000 1000 1001 1110 1111 0011 0010

Left shifting logically 8 times gives

R1= 1000 1001 1110 1111 0011 0010 0000 0000

2. LSR R1,#12

R1=0000 0000 1000 1001 1110 1111 0011 0010

Logically shifting it right 12 times gives,

R1= 0000 0000 0000 0000 0000 1000 1001 1110

3. ASR R2,R3

R2=1000 0000 0100 0101 0110 0111 0011 0000

Arithmetic right shift will do sign extension (the MSB is '1' here)

R2 = 1111 1111 1000 0000 0100 0101 0110 0111

4. ROR R2,R3

R2=1000 0000 0100 0101 0110 0111 0011 0000

Rotating it right eight times will make it

R2=0011 0000 1000 0000 0100 0101 0110 0111

5.2.1.6 MOV Instruction (MOV, MVN)

- The most frequently used instruction in any processor is the one that copies from a source to a destination.
- In ARM, the 'MOV' instruction does the copying of data to a destination register.
- The simplest formats of the MOV instruction are as follows:

1. MOV dest, src	;Move (copy) the contents of the source to the destination
2. MVN dest, src	;Move (copy) the negated value of source to destination
MOV R1, R3	;copy the value in R3 to R1
MVN R5, R9	;copy the negated value of R9 to R5
MOV R4,#45	;copy the immediate 8 bit number 45 to R4

EXAMPLE 1

What is the content of the destination registers after the execution of the following instructions?

R1 = 0

R2 = 0x896700FF

1. MOV R3, R1
2. MVN R4, R1
3. MVN R5, R2
4. MOV R6, #235

SOLUTION

1. R3 = 0

2. R4 = 0xFFFFFFFF

3. R5 = 0x7698FF00

4. R6 = 0x000000EB (235 in 32-bit hexadecimal format)

EXAMPLE 2

Find the result in the destination register after the execution of the following instructions, if:

R6 = 0xFFEE1100 and R2 = 8

1. MOV R10, R6, LSR #2

2. MVN R9, R6, LSL #4

3. MOV R8, R6, ASR R2

Given:

FF EE 11 00

11111111 11101110 00010001 00000000

1) MOV R10, R6, LSR #2

Before: 11111111 11101110 00010001 00000000

After LSR #2: R10 = 00111111 11111011 10000100 01000000 = 0x3FFB8440

2) MVN R9, R6, LSL #4

Before: 11111111 11101110 00010001 00000000

After LSL #4: 11111110 11100001 00010000 00000000

Before NOT: 11111110 11100001 00010000 00000000

After NOT: 00000001 00011110 11101111 11111111

R9 = 0000 0001 00011 110 1110 1111 1111 1111 = 0x011EEFF

3) MOV R8, R6, ASR R2 (R2 = 8)

Before: 11111111 11101110 00010001 00000000

After ASR #8: 11111111 11111111 11101110 00010001

R8 = 1111 1111 11111111 11101110 00010001 = 0xFFFFEE11

5.2.1.7 THE SUFFIX 'S'

- For any data processing instruction to have the capability to update the conditional flags, it must be suffixed with the character 'S'.
- If this suffix is not appended to the instruction, the updating of flags does not occur.
- For example, the MOV instruction may be appended as MOVS.
- Other instructions which we will see presently, may also be appended with the suffix S

IF R1=0, R2=0x896700FF,

What is the content of the destination registers after the execution of the following instructions? Which flags are updated for each case?

1. **MOVS R3,R1**
2. **MVNS R4,R1**
3. **MVNS R5,R2**

1. The destination register R3 contains 0x0. The Z flag is updated to 1, indicating that the destination register is zero.
2. The destination register R4 contains 0xFFFFFFFF. The N (Negative) flag is set to 1 indicating that the MSB of the destination is 1, that is, a negative number is in the destination register.
3. The destination register R5 contains 0x7698FF00. None of the conditional flags are set. N=0, as the MSB is not '1', because a positive number is in the destination register.

5.2.1.8 CONDITIONAL EXECUTION

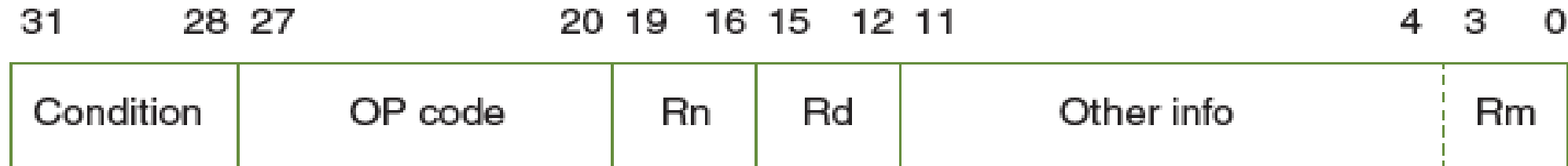
- One of ARM's special features is that it can execute any instruction conditionally.
- The word conditionally implies that the state of the condition flags are checked before it is decided whether the instruction should be executed or not.
- If the required condition is not met, the instruction becomes a NOP (No Operation).

LIST OF CONDITION CODES and the FLAGS UPDATED CORRESPONDINGLY

Table 5.3 List of condition codes

Code	Suffix	Flags	Meaning
0000	EQ	Z set	equal
0001	NE	Z clear	not equal
0010	CS	C set	unsigned higher or same
0011	CC	C clear	unsigned lower
0100	MI	N set	negative
0101	PL	N clear	positive or zero
0110	VS	V set	overflow
0111	VC	V clear	no overflow
1000	HI	C set and Z clear	unsigned higher
1001	LS	C clear or Z set	unsigned lower or same
1010	GE	N equals V	greater or equal
1011	LT	N not equal to V	less than
1100	GT	Z clear AND (N equals V)	greater than
1101	LE	Z set OR (N not equal to V)	less than or equal
1110	AL	(ignored)	always

Format of a Data Processing Instruction



- Figure shows the instruction format of a typical data processing instruction.
- Looking at Fig, we see that four bits are allocated to specify the condition.
The instruction produces a result by performing a specified arithmetic or logical operation on one or two operands. The first operand is always a register (Rn).
- The second operand may be a shifted register (Rm) or a rotated 8 bit immediate value.
- Rd is the destination register.
- The flag bits in the CPSR may be updated if the instructions has been appended with 'S'.
- Otherwise, the flags are not affected.

How Do We Use Conditional Execution?

Let us take a few examples.

`MOVEQ R2,R4`

The above instruction checks the flags associated with the condition ``EQ:(equal)`. Referring to Table 5.3, we see that the Z flag should be set ($Z=1$), is the condition specified.

After this instruction is fetched and decoded, the condition (whether Z flag is set) is checked before this instruction comes to the execution stage. If found to be true, the execution of copying the content of R4 to R2 is done. But if $Z=0$, the instruction simply becomes an NOP, that is, it does not get executed.

MOVHI R5,R6

Before it is decided whether these instructions should be executed or not, the flags associated with the 'HI' condition is looked at. Table 5.3 shows that $C=1$ and $Z=0$ are the conditions.

If a previous instruction had updated the flags and these conditions are met, the MOVHI instruction gets executed — otherwise it becomes a NOR

What Are The Pros And Cons Of Such Conditional Execution (Which Is Not Done In most Other ISAs)?

The plus points (pros) are:

- ❖ Most ISAs use conditions only with branch instructions. Here, conditions may be appended to any other instruction, and the condition evaluation hardware is shared for many instructions. The end result is that the effective number of instructions in the ISA becomes more than as seen directly. This is achieved without additional hardware.
- ❖ Codes become dense and branching is avoided to a great extent
- ❖ By avoiding branching (unless absolutely necessary), pipeline stalling is prevented

What Are The Pros And Cons Of Such Conditional Execution (Which Is Not Done In most Other ISAs)?

The minus points (cons) is:

- When an instruction becomes a NOP, one cycle time is wasted.

- This is because, even though execution of the instruction is not done, 'fetching and decoding' cannot be avoided.

5.2.1.9 ARITHMETIC AND LOGICAL INSTRUCTIONS

The general format is

$\langle \text{opcode} \rangle \{ \text{cond} \} \{ S \} \text{Rd}, \text{Rn} \langle \text{Op2} \rangle$ where, Rd is the destination register, and Rn and Op2 are the source operands.

5.2.1.9 ADDITION AND SUBTRACTION

lists of addition and subtraction instructions of this ISA

Table 5.4 List of ADD and SUBTRACT instructions

Operation	A typical instruction	Computation
Add	ADD R5, R0,R1	$R5 = R0 + R1$
Add with carry	ADC R5,R0,R1	$R5 = R0 + R1 + C$
Subtract	SUB R5,R0,R1	$R5 = R0 - R1$
Subtract with carry	SBC R5,R0,R1	$R5 = R0 - R1 - !C$
Reverse Subtract	RSB R5,R0,R1	$R5 = R1 - R0$
Reverse Subtract with carry	RSC R5,R0,R1	$R5 = R1 - R0 - !C$

Take a look at this code snippet

```
ADDS R2,R3,R4  
MOVCS R10,R2  
MOV R9,R2
```

What is done here?

The first line adds the content of R3 and R4 and saves the sum in R2. Since the suffix 'S' has been appended to the ADD instruction, the flags are updated. The next line has a condition 'CS' for checking the content of the carry flag (Ref Table 5.3). If the addition has caused a 'carry', the second instruction is executed and the content of R2 is copied to R10. If the carry flag is not found to be set, the second instruction becomes a NOP and execution proceeds to the third instruction in which R2 is copied to R9. The end result is that R10 will have the sum of R3 and R4 only if the addition causes a 'carry'. Otherwise, only R9 will have this sum. Note that the third instruction is an unconditional one.

Table 5.5 List of logical instructions

Operation	A typical instruction	Computation
AND	AND R5, R0,R1	R5=R0 AND R1
OR	ORR R5,R0,R1	R5=R0 OR R1
EX-OR	EOR R5,R0,R1	R5=R0 EX-OR R1
Bit Clear	BIC R5,R0,R1	R5=R0 (AND NOT) R1

EXAMPLE

Find the output of the following instructions given that R2=0x0F, R1=0xF0

1. **AND R3,R1,R2**
2. **ANDS R3,R1,R2**
3. **BIC R5,R1,R2**

SOLUTION

1. **R3=0x00** The destination register has '0' as its content. But Z flag is not set.
2. **R3=0x00** Here the 'Z' flag is set, because the instruction has the 'S' suffix.
3. **R5= 0xF0** Here the operation is that, R1 does an AND operation with the complement of the second operand(R2). The result shows (as '1') which of the bits of the first operand (R1) are '0's.

BIC instruction $Rd = Rn \text{ AND } (\text{NOT Operand2})$

5.2.1.10 COMPARE Instruction

There are four instructions for comparison.

Table shows the list and the operations performed for each of them.

The important feature of these instructions is that they do not produce a result — they only update the flags.

Thus, the `S' suffix is not needed for them.

The format for these instructions is

`<opcode>{cond}Rn, Operand2`

Table 5.6 List and operations of compare instructions

Operation	A typical instruction	Computation
Compare	CMP R1,R2	$R1 - R2$
Compare negated	CMN R1,R2	$R1 + R2$
Test on Equal	TEQ R1,R2	$R1 \text{ XOR } R2$
Bit test	TST R1,R2	$R1 \text{ AND } R2$

Table 5.7 Flag setting after a compare instruction

If	C	Z	N
$R1 > R2$	1	0	0
$R1 < R2$	0	0	1
$R1 = R2$	1	1	0

PROBLEM

R1=0x0,R2=0xF

What is the result and relevance of the following two instructions?

1. **TST R1,R2**
2. **TEQ R2,#0x0F**

SOLUTION

1. Since the ANDing of the two registers produces a '0', the Z flg is set.
Here the second operand R2 is said to be a 'mask' and it verifies if any bit of the first operand has at least one '1'. The fact that $Z=1$, makes it clear that R1 does not have a '1' in the lowest 4 bit positions.
If $R1=01$ and $R2=0xFF$, the Z flag is not set. Thus, it is confirmed that the R1 has at least one '1'.
2. Since the Ex-oring of the two registers produce a zero, the Z flag is set. TEQ does exclusive ORing which tests for equality. If both the operands are equal, the Z flag is set. It verifies if the value in a register is equal to a specified number which is the second operand.

PROBLEM

Now let us use some arithmetic with shifts and rotates (All results of shifts must remain within the 32 bit range of the register size).

1. **MOV R10,R10,LSL #2**
2. **MOV R2,R2,LSR #2**
3. **MOV R5,R5,ASR #2**
4. **ADD R5,R5,R5,LSL #4**
5. **RSB R3, R4, R4, LSL #4**
6. **SUB R1, R1, R1,LSL #3**

SOLUTION

1. $R10 = R10$ left shifted by 2, which is $2^2 R10 = 4R10$
2. $R2 = R2$ divided by $2^2 = R2/4$
3. $R5 = R5$ divided by 2^2 , but signed.
4. $R5 = R5 + R5 \times 2^4 = R5 + 16R5 = 17R5$
5. $R3 = 16R4 - R4 = 15R4$
6. $R1 = R1 - 8R1 = -7R1$

PROBLEM

How can 64 bit addition be done using the 32-bit ARM processor?

Let the 64 bit numbers be in two sets of registers say
R1, R0 (R0 holding the lower 32 bits)
R3, R2 (R2 holding the lower 32 bits)
The program lines are

```
ADDS R2,R2,R0  
ADC  R3,R3,R1
```

The 64-bit sum will be R3 and R2

If the result is expected to be more than 64 bits, the code is

```
ADDS R2,R2,R0  
ADCS R3,R3,R1
```

The sum will then be in the Carry bit (MSB) and R3,R2. It will be a 65-bit number.

5.2.1.11 MULTIPLICATION

There are two basic multiplication instructions.

One is 'Multiply' and the other is 'Multiply and Accumulate'

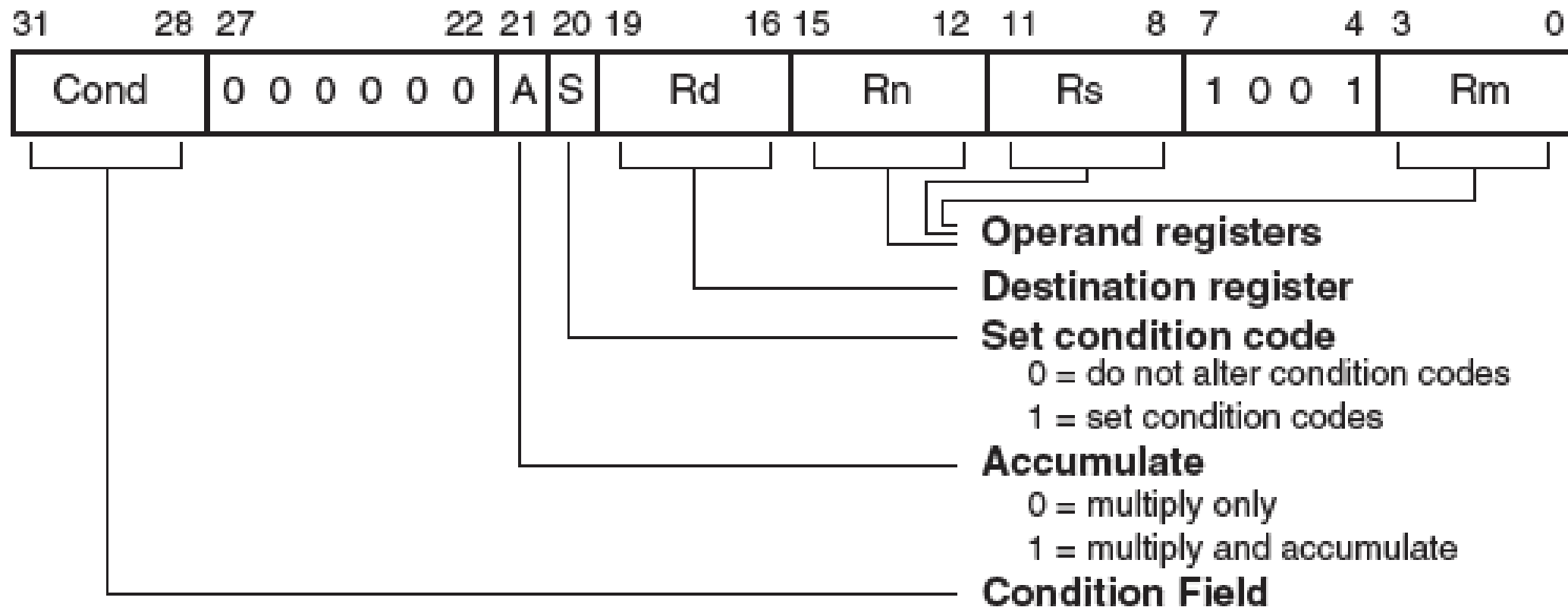
The word 'Accumulate' indicates 'Addition'

Instruction format for the both is:

$\text{MUL}\{\langle\text{cond}\rangle\}\{S\} \text{ Rd, Rm, Rs} \quad ; \text{Rd} = \text{Rm} * \text{Rs}$

$\text{MLA}\{\langle\text{cond}\rangle\}\{S\} \text{ Rd, Rm, Rs, Rn} \quad ; \text{Rd} = (\text{Rm} * \text{Rs}) + \text{Rn}$

Figure 5.8 Instruction format of multiplication instructions



PROBLEM

Examples

MUL R1,R2,R3

;R1=R2xR3

MULS R1, R2, R3

;R1=R2xR3 and flags are updated

MULEQ R1,R2,R3

;R1=R2xR3 executed only if Z flag is set

MLA R4, R3, R2, R1

;R4 = R3xR2+R1

MLAEQS R1,R2,R3,R4

;R1:=R2*R3+R4 –executed only if Z=1 ;On execution, flags are updated

Restrictions on the use of the multiply instructions:

- The source and destination registers are 32 bits in length. If the product is longer than 32 bits, only the lower bits are preserved in the destination register.
- Immediate data cannot be used as a source operand
- The PC (R15) cannot be used as any of the registers
- Operands can be considered signed or unsigned. The user must be able to interpret correctly, the results of the multiplication.

Table 5.8 List of extended multiplication instructions

Mnemonic	Description	Purpose
UMULL{cond}{S} RdLo,RdHi,Rm,Rs	Unsigned Multiply Long	$32 \times 32 = 64$
UMLAL{cond}{S} RdLo,RdHi,Rm,Rs	Unsigned Multiply & Accumulate Long	$32 \times 32 + 64 = 64$
SMULL{cond}{S} RdLo,RdHi,Rm,Rs	Signed Multiply Long	$32 \times 32 = 64$
SMLAL{cond}{S} RdLo,RdHi,Rm,Rs	Signed Multiply & Accumulate Long	$32 \times 32 + 64 = 64$

5.2.1.12 DIVISION

- ❖ There is no explicit divide instruction in this ISA
- ❖ Division can be accomplished by repeated subtraction
i.e .

```
DIV      MOV QUOTIENT,#0
LOOP3    SUBS DIVIDEND,DIVIDEND,DIVISOR
          ADDPL QUOTIENT,QUOTIENT,#1
          BPL LOOP3
          ADDMI REMAINDER,DIVIDEND,DIVISOR
          BX LR
```

5.2.1.13 BRANCH Instruction

Branch instructions are the ones that give the power of decision making to a computer.

For ARM, there are no jump or call instructions as in the case of other processors.

The branch instruction with the mnemonic 'B' suffices for all jumping operations and procedure calls.

5.2.1.13.1 BRANCH (B)

The general format of a branch instruction is 'B label'
'label' corresponds to the address of the target location.

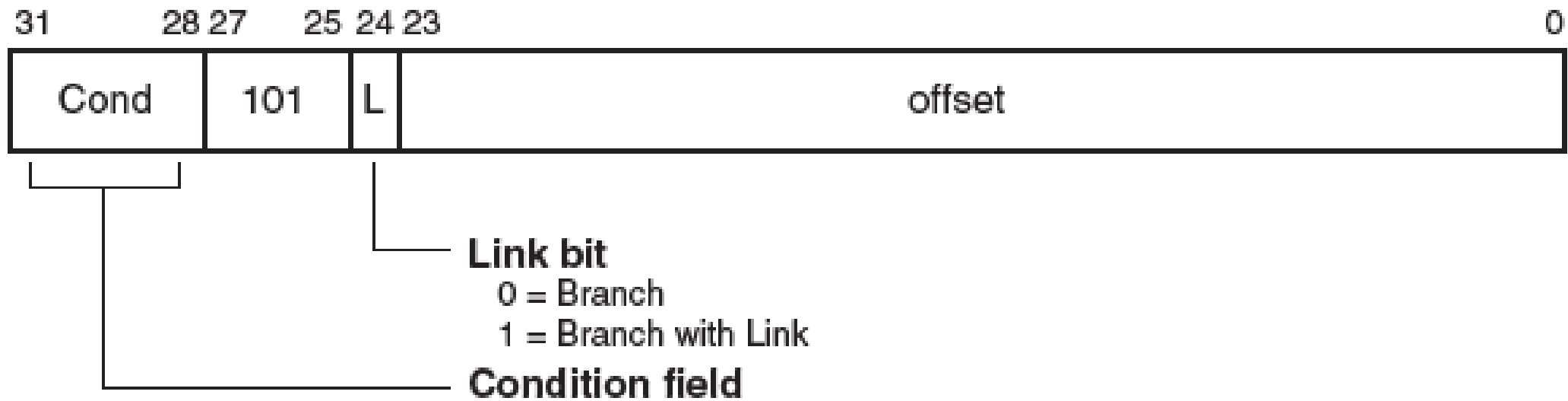
Condition codes may be appended to make it a conditional branch instruction.

The assembler finds the target address with the use of the offset specified in the instruction.

Table 5.9 Branch instructions

Mnemonic	Instruction
B	Branch
BL	Branch and link
BX	Branch and Exchange
BLX	Branch Exchange with link

Figure 5.9 Format of branch instruction



5.2.1.13.2 BRANCH With LINK (BL)

- This instruction saves the current PC into the Link register (R14).
- Thus, BL facilitates subroutine/procedure calls.
- When this instruction is encountered, branching occurs to the target address and the 'return address' (the address to which control must come back after the procedure is ended), is saved in the link register.

5.2.1.13.3 BRANCH With LINK Exchange (BLX)

BLX in ARM BLX = Branch with Link and Exchange

It's used for:

1. Function call (like BL)
2. Switching instruction set (ARM $\leftrightarrow$ Thumb)

Syntax : BLX label

1. Branch

- Jumps to target address (label or register)

2. Link

- Stores return address in LR (R14)

3. Exchange

- Switches instruction set depending on LSB of target address(0/1)
or(ARM/Thumb mode)

Unit -IV

Chapter-5

Assembly language programming

5.3 Assembly language programming

- Now we can start assembly programming using the Keil Microvision V assembler.
- The step-by-step method of using this IDE for Assembly Language

5.3.1 Assembler Directives

For each assembler, there are 'directives'.

They are specific for an assembler and are non-executable statements, which means that they don't cause any 'execution' by the processor.

Directives are related solely to the assembler.

We need a few directives of the Keil Assembler to get started in assembly programming.

5.3.1.1 AREA

The “**AREA**” directive instructs the assembler to assemble a new code or data section".

The above is a statement given in the user manual of Keil Microvision.

It implies that the assembler understands two types of information —code and data.

Data is usually stored in R/W memory (RAM) while code is in Read only memory (ROM).

The assembler requires the names of these areas of code or data.

- Typical statements that use the AREA directive are

AREA LINEAR, CODE, Read only

AREA NOTE, DATA, R/W

- It is not mandatory to use the words 'Read Only' and 'R/W' because by default, data is in R/W memory and code is in Read Only Memory.
- Code is always in ROM, though data is also allowed to be stored there.
- As a matter of fact, the R/W memory is volatile and is used only for intermediate storage of data in the course of instruction execution

5.3.1.2 ENTRY

The ENTRY directive marks the first instruction to be executed within an application.

Because an application cannot have more than one entry point, the ENTRY directive can appear in only one of the source modules.

5.3.1.3 END

This directive is the last line in an assembly module and it indicates that the assembler need not read beyondthat line.

5.3.1.4 DCB, DCW and DCD

The ARM processor has a word length of 32 bits, but it can handle data of 8 bits (byte) and 16 bits (half word) also.

The Keil assembler, however, uses the notations of byte, word and double word for 8 bits, 16 bits and 32 bits, respectively.

The directives used are DCB for data byte, DCW for data word and DCD for data double word.

The following lines show how these directives are used.

These directives are used by the assembler to store data in ROM.

Example

BAG DCB 0x32, 56, 0x7E

BAG DCW 0x2234, 0x ED37, 0x9067

BOG DCD 0x DE345678, 0x34009812

The first line says that three bytes are to be stored in contiguous locations with labels, RAG, RAG+1, and RAG+2. Recollect that each address always corresponds to one byte of data. The second line indicates that 16 bit data is being stored in locations BAG, BAG+2 and BAG+4. The third line states that 32 bit data is to be stored in addresses with labels BOG and BOG+4.

5.3.1.5 equ

The EQU directive is for equating a label to a constant.

When the label is encountered by the assembler during the process of assembly, it is replaced by its actual value.

Examples of the usage of this directive are

- **RAPID EQU 65**
- **STRT_ADDR EQU 0x84000000**

Write an ALP (assembly language program) to add the first 5 natural numbers. The sum must be in a register.

AREA NAT, CODE, READONLY

ENTRY

MOV R0,#0 ;R0=0

MOV R1,#0 ;R1=0

BACKK ADD R0,R0,#1 ;Increment R0 by 1

ADD R1,R1,R0 ;Add the numbers.The sum is in R1

CMP R0,#5 ;check if R0=5

BNE BACKK ;repeat if R0 is not equal to 5

GO B GO ;continue branching to GO

END ;end of the assembly module

Write and test a program for adding the first ten odd numbers (starting from 1)

```
                AREA ODD, CODE, READONLY
ENTRY
    MOV R1,#1    ;R1=1
    MOV R2,#9;R2=9, the counter
    MOV R3,#1    ;R3=1
BACKK ADD R3,R3,#2    ;R3 has the odd numbers
      ADD R1,R1,R3    ;R1 contains the final sum
      SUBS R2,R2,#1    ;R2 is the counter for 10 numbers
      BNE BACKK ;repeat the addition until R2=0
GO      B GO
END
```

5.4 Accessing memory

We already know that ARM is a load-store architecture.

This means that only the load and store instructions access memory directly.

There are load/store instructions with single registers, as well as with multiple registers.

5.4.1 Single register data transfer

- The word 'load' indicates that data from memory is to be moved to a register. 'Store' means that data from a register is saved in memory
- The format for a load/store instruction is
LDR/STR {<cond>}<Rd>, <addressing mode>
- The operation of load/ store has a register Rd in which the data is placed. The 'addressing mode' implies that an effective address is generated by address calculation.

Example

LDR R2, [R7] ; copy into R2, the content of memory, as pointed by R7

STR R2, [R7] ; store the content of R2, into the address pointed by R7

The address of the memory location for the load/store operation is placed in a register, called the base register. The address may be generated and put into the base register in various calculation formats using shifting, adding offsets, summing the contents of more registers etc.

Find the effective address of memory as specified by the following instructions

1. LDR R4, [R7, ROR #4]
2. STR R7, [R3, R4, LSL #2]
3. LDR R6, [R2, #4]
4. STR R8, [R1, R2, ASR #6]
5. LDR R1, [R7, R6]

- The effective address is $R7 = R7$ rotated right 4 times.
- The effective address is $R3 = R3 + R4$ rotated left logically 2 times.
- The effective address is $R2 = R2 + 4$
- The effective address is $R1 + R2$ right shifted (arithmetically) 6 times
- The effective address is $R7 = R7 + R6$

5.4.2 Loading/Storing data of different lengths

- Data to be loaded or stored may be in bytes, half words (16 bits) or words (32 bits).
- Data may be signed or unsigned, as well.
- To make this explicit, the load and store instructions may be appended with the characters, B for byte, H for half word and S for signed.

Load/Store instructions

Instruction	Meaning
LDR	Load Word
LDRH	Load Half Word
LDRSH	Load Signed Half Word
LDRB	Load Byte
LDRSB	Load Signed Byte
STR	Store Word
STRH	Store Half Word
STRB	Store Byte

How is data accessed from memory with the following instructions, given that R7=0x00004400 and that memory contents are as given in Table (assume the little endian scheme)?

1. **LDR R3, [R7]**
2. **LDRB R3,[R7]**
3. **LDRH R3, [R7]**
4. **LDRSH R3, [R7]**

Address	Data
0x00004400	89
0x00004401	E6
0x00004402	F3
0x00004403	94

LDR R3, [R7]

This instruction pertains to a word (32 bits) of data to be loaded into the destination register. After the execution of the above instruction, R3=0x94F3E689.

LDRB R3, [R7]

After the execution of the above instruction, R3=0x00000089. This is because only a byte of the data will be taken from memory and so the lowest byte of the word (in the lowest address) pointed by R7, is loaded to R3.

LDRH R3, [R7]

After the execution of the above instruction, R3=0x0000E689. This is because the half word of the data will be taken from memory and the lower 16 bits of the word pointed by R7, is loaded to R3.

LDRSH R3, [R7]

After the execution of the above instruction, R3=0xFFFFE689. The lower half word of the data is 0xE689. The instruction LDRSH considers this to be a 'signed halfword' and when loading it to a 32-bit register, the sign bit is extended to the upper 16 bits.

The content of registers, R6 and R3, and memory are as given below. R6= 0x40002200, R3=0x3498CE56

Show the content of memory after the execution of the following instructions

- STRB R3, [R6]**
- STRH R3, [R6]**
- STR R3, [R6]**



Address	Data
0x40002200	00
0x40002201	00
0x40002202	00
0x40002203	00

- **STRB R3, [R6]**

Address	Data
0x40002200	56
0x40002201	00
0x40002202	00
0x40002203	00

- **STRH R3, [R6]**

Address	Data
0x40002200	56
0x40002201	CE
0x40002202	00
0x40002203	00

- **STR R3, [R6]**

Address	Data
0x40002200	56
0x40002201	CE
0x40002202	98
0x40002203	34

5.4.3 Pre Indexing, Post Indexing and Write Back

- Consider the Instruction

STR R0, [R7,#2] and LDR R0,[R7,ROR #4]

- In the first instruction, it calculates the effective address as $R7 = R7 + 2$.
- In the second instruction, it calculates the address as $R7 = R7 \text{ ROR } \#4$.

What is the content of R7 after the execution of previous instruction?

- The content of R7 remains the same as it was before the instruction execution.
- Means the address calculation before, no change in R7

Write Back(!):

- If it necessary to save the effective address into the base registers,
- Instruction should have the concept of “ Write Back”.
- Means the instruction need to have with the sign “ !”.
- Write back instruction is:

STR R0, [R7,#2]! and LDR R0,[R7,ROR #4]!

Examples

What is the content of the base register R6 after the execution of each of the following instructions, given that R6=0x44001100 and R2 = 0x02.

1. LDR R2,[R6,#0x0056]!
2. STR R1,[R6,R2]!
3. LDR R3,[R6,LSL #4)

1 . LDR R2,[R6,#0x0056]!

The effective address is calculated as $R6 = R6 + 0x0056 = 0x44001156$, because of write back, this number will be saved in R6 after instruction execution.

2. STR R1,[R6,R2]!

The effective address is calculated as $R6 = R6 + R2 = 0x44001102$ because of write back, this number will be saved in R6 after instruction execution.

3. LDR R3,[R6,LSL #4]

The effective address is calculated as $R6 = R6, \text{LSL}\# 4$. Because the instruction does not have the write back feature (!), after instruction execution, R6 will retain its previous content of 0x44001100.

5.4.4 Post Indexing

- In this mode, the effective address calculation is done after the execution of the specific instruction.
- Thus, write back is inherently implied.
- Take the case of the instruction `LDR R2, [R7], #2`
- Here the data pointed by the content of R7 is first copied to R2.
- After that, the content of R7 (which is the base register) is changed to $R7 + 2$.

5.4.5 Immediate mode of addressing

- This is the mode when an immediate data is used as an operand.
- Consider the instruction

`MOV R1, #10` ; move data 10 to R1

`ADD R4, R2, #3` ; Add 3 to the content of R2, and save the sum in R4

These instructions seems to be ok. But there is an unseen issues involved

What is the biggest constant that can be used in the immediate mode?

- All instructions of ARM are to have same size 32 bits
- If an immediate data is to be included, it must fit in the 32-Bit size allocated for a data processing instruction.
- Means the larger sizes of constants cannot be used in immediate data.

- The figure 5.10 shows the format of a typical data processing instructions of ARM.
- If bit 25 is '0', it is understood that the operand2 is a register.
- If bit 25 is '1', the operand2 is immediate data.
- 12bit space is allocated for operand2.



Figure 5.10 Format of an immediate mode instruction

- The 12bit space is arranged as a four bit rotate value and an eight bit immediate value as shown in figure 5.11.
- The rotation bit field is only 4 bits .
- Internally this rotation field is multiplied by 2, and thus becomes the case of an 8 bit number multiplied an even number of times to get a new number.

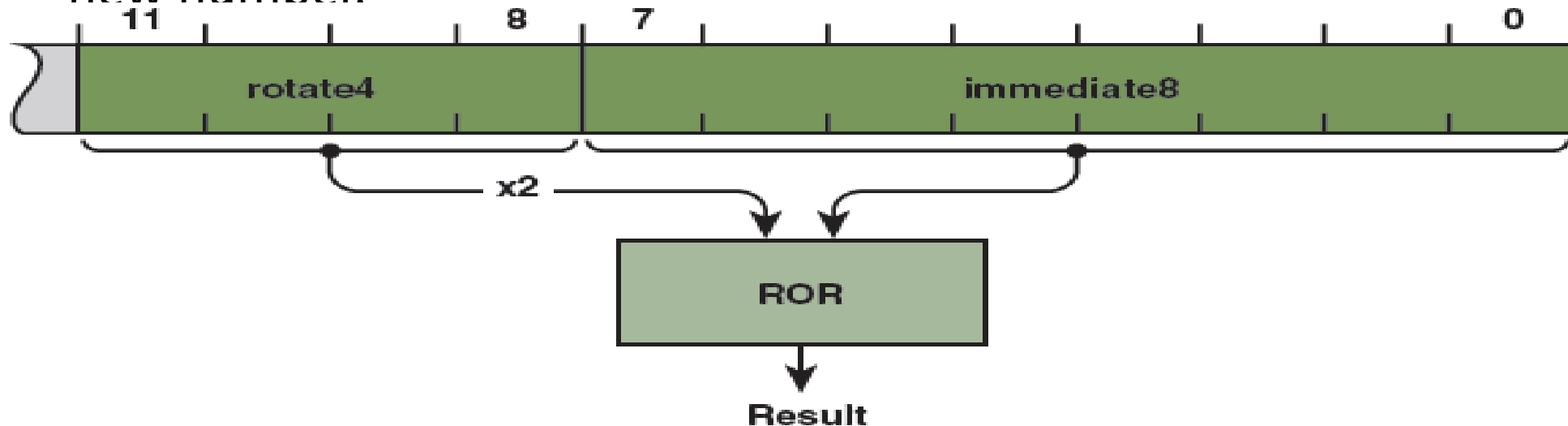


Figure 5.11 Rotate operation for handling immediate data

Perform the ARM rotation scheme for the number 0xE0.

The number in 32 bit space is written as

0000 0000 0000 0000 0000 0000 1110 0000

The number rotated 4 times to the right becomes

0000 0000 0000 0000 0000 0000 0000 1110 i.e. 0x0E

The number rotated 6 times to the right becomes

1000 0000 0000 0000 0000 0000 0000 0011 i.e. 0x80000003

Table 5.12 Constants generated by rotation schemes (Courtesy: ARM ASSEMBLY LANGUAGE: William Hohl)

Decimal values	Equivalent hexadecimal	Step between values	Rotate
0 – 255	0 – 0xff	1	No rotate
256,260,264, ...,1020	0x100 – 0x3fc	4	Right by 30 bits
1024,1040,1056, ..., 4080	0x400 – 0xff0	16	Right by 28 bits
4096,4160,4224, ...,16320	0x1000 – 0x3fc0	64	Right by 26 bits

Find the constants obtained by rotation, for the following cases.

1. Rotate 0x30, to the right 28 times
2. Rotate 0x96, to the right 14 times
3. Rotate 0x5C, to the right 10 times
4. Rotate 0x09, to the right 15 times
5. Rotate 0xFC, to the right 6 times

SOLUTION

1. 0000 0000 0000 0000 0000 0000 0011 0000

Rotating 28 times to the right (which is the same as rotating 4 times to the left) is:

$$0000\ 0000\ 0000\ 0000\ 0000\ 0011\ 0000\ 0000 = 0x00000300$$

2. 0000 0000 0000 0000 0000 0000 1001 0110

Rotating it to the right 14 times will make it

$$0000\ 0010\ 0101\ 1000\ 0000\ 0000\ 0000\ 0000 = 0x02580000$$

3. The rotation generates the number 0x17000000
4. The rotation generates the number 0x00120000
5. The rotation generates the number 0xF0000003

- When we want larger constant, it is not possible to write it in an instruction like MOV R1, #n, where n is constant
- Need to use the immediate mode in such a way that the constant that we need is obtained by the ARM rotation scheme
- Not possible by programmer to find out exact 8-bit number which will produce a bigger size constant
- So Assembler will come to an assistant here.
- for that, a pseudo instruction is provided
- The format is LDR Rd, =constant ; where Rd is destination register

ex: LDR R1, =0x44444444

- But all constants are not obtainable by such a scheme.
- Consider 0x77777777.
- This constant cannot be generated in this way.
- In such a case, the assembler is used to store the required constant in memory and take it from memory using the LDR instruction.
- The area in memory where such constants are loaded is called a “literal pool”.
- programmer need only write the pseudo instruction.
- The assembler will handle the matter in an efficient manner.
- The sequence of actions that follow is highlighted below.
- Take the case of needing a constant 0x77777777.
- The pseudo instruction we write is

LDR R1 ,=0x77777777

AREA IMMID,CODE,READONLY

ENTRY

LDR R3, = 0x6D0000

LDR R4, = 0X77777777

ADD R5,R3,R4

GO B GOEND

Space for literal pool is:

- After the END directive, so that it can be stored in ROM in the space after the code.
- If the program is large and the pseudo instruction for it is far away from the end of the program, the literal pool may be beyond the range of +/-4KB.
- In such cases a directive LTORG(Origin of literal pool) is used.
- This directive may be included in the program to specify where we want the literal pool to be.

```
                AREA IMMID,CODE,READONLY
ENTRY
                LDR R3, = 0x6D0000
                LDR R4, = 0x77777777
                ADD R5,R3,R4
LTORG
                .....
                .....
                .....
                .....
                .....
GO B GO
END
```

Placing LTORG in this place gives a warning message 'Literal pool in executable area'.

```

                AREA IMMID,CODE,READONLY
ENTRY
                LDR R3, = 0x6D0000
                LDR R4, = 0x77777777
                ADD R5,R3,R4
                B  LUN
                LTORG
LUN             .....
                .....
                .....
                .....
                .....
                .....
                .....
GO              B GO
END.

```

- Literal pool are to be place in location in such a way that the processor does not consider them as instructions.
- They are best placed after unconditional branch instructions.

5.4.6 Read Only and Read/Write Memory

- We know that ROM is for code and RAM is for temporary storage of data.
- It is important to realize that data in RAM is accessed using instructions of the processor
- while data is placed in ROM using assembler directives like DCD, DCB etc.
- From ROM data can be brought to RAM by first loading it to registers using Load (LDR) instructions
- Later store them using STR instructions

Block data transfer using multiple registers:

- This ISA allows multiple registers to be loaded from contiguous memory locations, with the use of just one instructions.
- Similarly ,the content of Multiple registers may be saved into consecutive locations in memory. Such instructions are referred to as multiple load/store instructions.

LDM {cond}address-mode Rn{!},reg-list{^}

- Cond: is an optional condition
- address-mode: specifies how the memory is sequenced.
- Rn: is the base register for the load operation.
- ! : Specifies base register write back i.e the address in the base register is updated after the transfer.(if the base register is in the register list then no need to specify write back)
- Reg-list: List of symbolic register names and register ranges enclosed in braces. There must be at least one register in the list.
- ^ : This is not to be used in the user/system mode. It is meant for use in the privileged mode.

Addressing modes of the multiple load/store instructions:

- These modes indicate how to change the memory address after each word transfer.
- The addressing modes are:
 - IA:Increment After
 - IB :Increment before
 - DA:Decrement After
 - DB:Decrement Before

Instructions are

LDMIA R7! {R0-R5}

5.4.7.2 LDMIA R7! {R0-R5}

Let us look at a sample Load instruction

The operation will proceed as follows:

1. R7 is the base address, which means that it contains the address of memory to be accessed.
2. First the content of the address pointed by R7 is loaded to R0.
3. Since 'IA', that is 'Increment after', is the addressing mode, the content of R7 is incremented by 4. **This happens mandatorily, as only word (32 bit) transfers are supported by LDM and STM instructions.**
4. The content of the address pointed by the new value of R7 is loaded to R1.
5. The content of R7 is incremented again, and loading to R2,R3, R4 and R5 occurs.
6. Because of the write back feature (!), at the end of the operation, the value in R7 is the final updated address.

Note: Consider the instruction **LDmia R7!,{R10,R1-R2}**. It might seem that the first data is to be loaded in R10, but that is not correct. Whatever be the order in which the register list is written, the processor operation stipulates that data from the lowest address will always be loaded to the lowest register in the list, that is, R1 in this case.

If the instruction is **LDMDa R7!,{R10,R1-R2}**, the same logic has to be applied. R10 will be loaded with the data in the initial address in R7, because that is the highest address. As the address decrements after each operation, the registers R2 and then R1 is loaded.

The STM instruction:

Syntax: STM{cond} address-mode Rn!, reg-list{^}

- The operation is just the reverse of the LDM instruction.
- Here the contents of the registers are stored in memory location indexed by the base register.

STMDB R1!,{R7,R2-R4} if R1=0X44400088

- The base register is R1, it contains the number 0X44400088. Since the DB suffix is used, the R1 register is decremented by 4 and it contains the number 0X44400084.
- The data in R7 is saved to this location.
- The content of R1 is decremented by 4 and the data in R4 is saved in this address.
- These operations of decrementing the base register is repeated for storing the data in R3 and R2.
- At the end of the execution of the instruction, the value in R1 is the final updated value because write back is stipulated.

5.4.7.4 Realization of stacks

In this ISA, there is no PUSH and POP operation that we normally associate with stacks. Instead, the LDM and STM instructions are used for POP and PUSH operations, respectively. We know that with LDM, the content of a number of locations in memory can be popped into registers. Similarly, STM suffices for the pushing of the contents of registers into memory.

We know that a stack is a data structure which we can define as we wish. There can be decending/ascending stacks which may be full or empty as the following definitions indicate.

1. **Descending or ascending:** The stack grows downwards, starting with a high address and progressing to a lower one (a descending stack), or upwards, starting from a low address and progressing to a higher address (an ascending stack).
2. **Full or empty:** The stack pointer can either point to the last item in the stack (a full stack), or the next free space on the stack (an empty stack).

Table 5.13 illustrates the way these stacks are realized.

The Procedure Call Standard for the ARM Architecture (AAPCS) always uses a full descending stack.

When using a stack, the base register is SP. Writeback is mandatory. The following instructions define push and pop for such a stack (Full Descending):

STMFD SP!,{R0–R5} ;Push onto a full descending stack
STMDB SP!,{R0–R5} ;‘Decrement Before’ is also for pushing to a full descending stack
LDMFD SP!,(R0–R5) ;Pop from a full descending stack

Table 5.13 Types of stacks as addressing modes to be used for them

Stack type	Push	Pop
Full Descending	STMFD(DB)	LDMFD(IA)
Full Ascending	STMFA(IB)	LDMFA(DA)
Empty Descending	STMED(DA)	LDMED(IB)
Empty Ascending	STMEA(IA)	LDMEA(DB)

LDMIA SP!,{R0–R5} ;Increment after is also for popping from a full descending stack.
 Before we use a stack, the stack pointer has to be initialized to the top of the stack to the topmost location in memory that we decide our stack to be.

What does this program do?

```
AREA    FDSTCK,CODE,READONLY
```

```
ENTRY
```

```
    LDR  SP,=0X40000030
```

```
    MOV R0,#5
```

```
    MOV R1,#6
```

```
    MOV R2,#7
```

```
    STMDB SP!, {R0-R2}
```

```
    LDMIA SP!, {R9,R4-R5}
```

```
GO    B      GO
```

```
    END
```

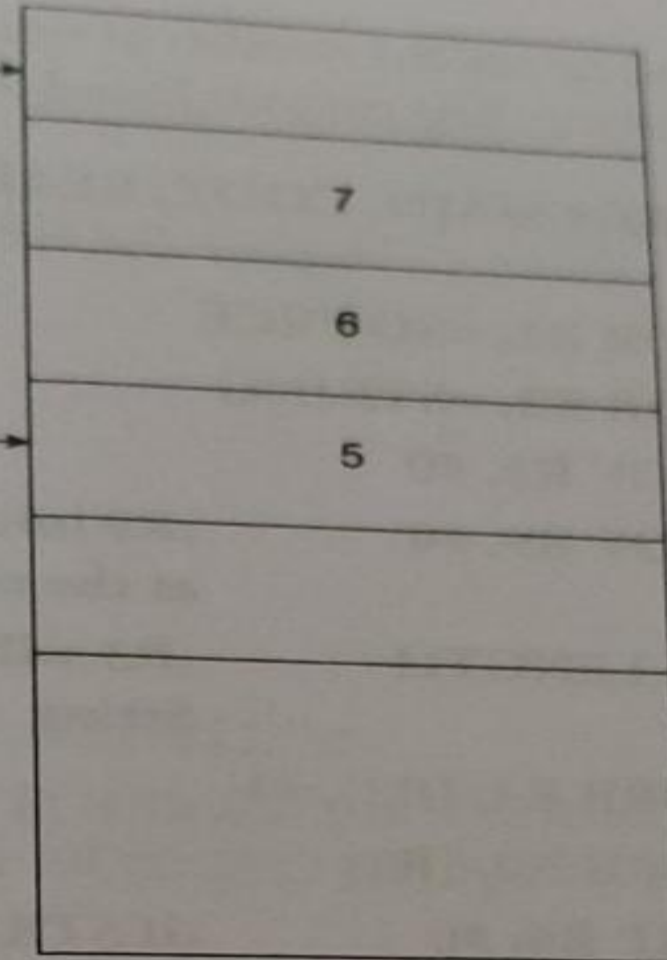
1. SP is initialised to **0x40000030**.
2. Values 5,6, and 7 are copied to registers R0, R1 and R2.
3. The content of these registers are pushed to stack.
4. The content of the highest register (R2) is stored in the highest memory location.
5. From the stack, the numbers are loaded (popped) to R9,R4 and R5.
6. Note that R9 will now have '7', which was stored in the highest memory location.
7. Figure 5.12 shows the value of SP as being 0x40000030 before the pushing operation using the STMDB instructions. After three words are pushed onto the stack, SP=0x40000018.

Before STMDB

SP = 0 × 40000030

SP = 0 × 40000018

After STMDB



5.12 Pushing into the stack